

Problem 1

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `~/Documents/Spring\_2026/BEE4850/hw/hw02-avinasharavind`

```
using Random # random number generation and seed-setting
using DataFrames # tabular data structure
using CSV # reads/writes .csv files
using Distributions # interface to work with probability distributions
using Plots # plotting library
using StatsBase # statistical quantities like mean, median, etc
using StatsPlots # some additional statistical plotting tools
using Optim # optimization tools
using LaTeXStrings # latex formatting for plot strings/
```

1.1

```
chicago = CSV.read("data/chicago.csv", missingstring="NA", DataFrame)
```

	Column1	death	pm10median	pm25median	o3median	so2median	time	tmpd
	Int64	Int64	Float64?	Float64?	Float64	Float64?	Float64	Float64
1	1	130	-7.43354	<i>missing</i>	-19.5923	1.92804	-2556.5	31.5
2	2	150	<i>missing</i>	<i>missing</i>	-19.0386	-0.985563	-2555.5	33.0
3	3	101	-0.826531	<i>missing</i>	-20.2173	-1.89142	-2554.5	33.0
4	4	135	5.56646	<i>missing</i>	-19.6757	6.13934	-2553.5	29.0
5	5	126	<i>missing</i>	<i>missing</i>	-19.2173	2.27846	-2552.5	32.0
6	6	130	6.56646	<i>missing</i>	-17.634	9.85858	-2551.5	40.0
7	7	129	-0.433544	<i>missing</i>	-15.3744	-5.81899	-2550.5	34.5
8	8	109	-5.43354	<i>missing</i>	-12.1705	-5.10794	-2549.5	29.0
9	9	125	-0.571429	<i>missing</i>	-20.0923	0.182237	-2548.5	26.5
10	10	153	<i>missing</i>	<i>missing</i>	-18.5803	-2.04693	-2547.5	32.5
11	11	124	-19.4335	<i>missing</i>	-5.71219	-1.601	-2546.5	29.5
12	12	111	-15.4335	<i>missing</i>	-15.6289	2.93793	-2545.5	34.5
13	13	104	11.5665	<i>missing</i>	-17.0455	3.6418	-2544.5	34.0
14	14	118	1.56646	<i>missing</i>	-18.6289	6.18347	-2543.5	37.5
15	15	109	-7.25602	<i>missing</i>	-18.7122	-1.14142	-2542.5	32.5
16	16	125	-22.4335	<i>missing</i>	-15.0039	-3.15381	-2541.5	25.0
17	17	128	<i>missing</i>	<i>missing</i>	-17.3789	1.80724	-2540.5	27.0
18	18	141	-2.43354	<i>missing</i>	-17.0039	0.562692	-2539.5	17.5
19	19	130	-9.43354	<i>missing</i>	-7.634	-0.718613	-2538.5	23.0
20	20	133	-3.43354	<i>missing</i>	-18.4136	0.681604	-2537.5	20.5
21	21	115	-4.17241	<i>missing</i>	-16.9933	3.65346	-2536.5	22.0
22	22	121	10.5665	<i>missing</i>	-12.6289	-1.26891	-2535.5	19.5
23	23	107	13.5665	<i>missing</i>	-7.92567	-2.26891	-2534.5	2.5
24	24	123	-3.43354	<i>missing</i>	-12.259	-1.72475	-2533.5	2.0
...	...	...	...	...	...	...	...	...

```
describe(chicago, :min, :q25, :median, :q75, :max, :mean, :nmissing)
```

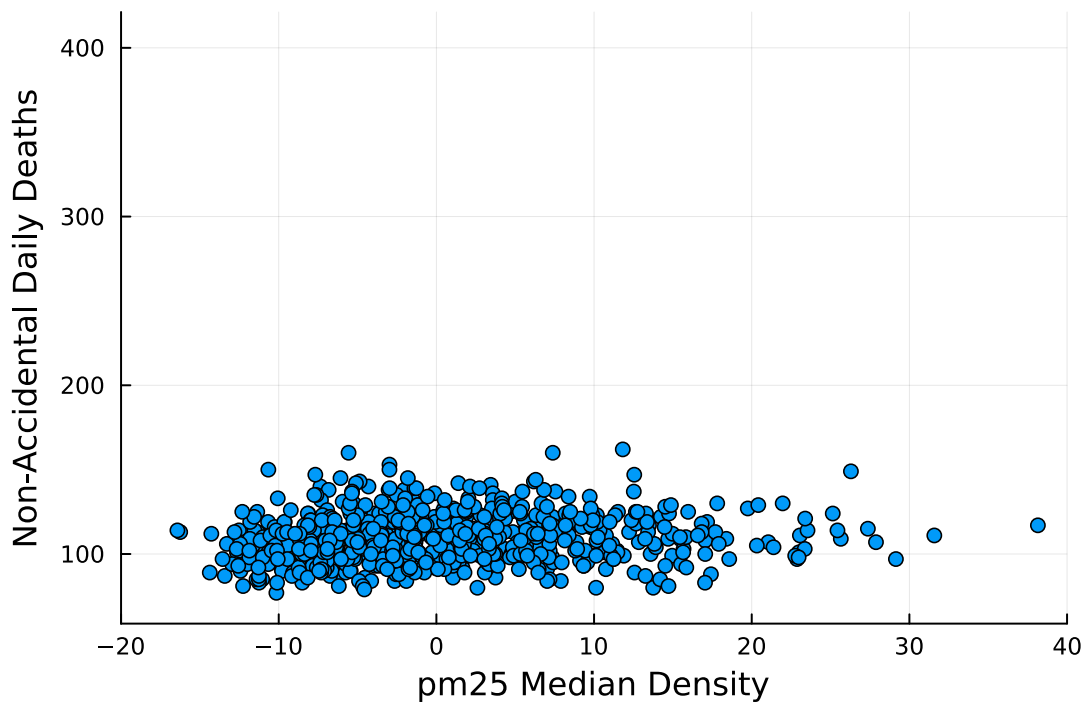
	variable	min	q25	median	q75	max	mean	nmissing
	Symbol	Real	Float64	Float64	Float64	Real	Float64	Int64
1	Column1	1	1279.25	2557.5	3835.75	5114	2557.5	0
2	death	69	105.0	114.0	124.0	411	115.419	0
3	pm10median	-37.3761	-13.1082	-3.53906	8.30292	320.725	-0.14639	251
4	pm25median	-16.4264	-6.58841	-1.32584	5.34377	38.1504	0.243053	4387
5	o3median	-24.7794	-10.2319	-3.32586	4.46821	43.6878	-2.17938	0
6	so2median	-8.2061	-2.68935	-1.21826	0.831593	28.9034	-0.636071	27
7	time	-2556.5	-1278.25	0.0	1278.25	2556.5	0.0	0
8	tmpd	-16.0	35.0	51.0	67.0	92.0	50.1933	0

```

plot(chicago.pm25median, chicago.death,
     st="scatter", xlim=[-20,40],
     xlabel="pm25 Median Density",
     ylabel="Non-Accidental Daily Deaths",
     title="Chicago pm25 Density vs. Daily Deaths",
     legend=false,
     rightmargin=10Plots.mm,
     leftmargin=5Plots.mm,
     bottommargin=5Plots.mm
    )
savefig("figs/1_1_pm25.png")
plot!() #To show plot in notebook

```

Chicago pm25 Density vs. Daily Deaths



```

plot(chicago.o3median, chicago.death,
     st="scatter", xlim=[-25,45],
     xlabel="Ozone Median Concentration",
     ylabel="Non-Accidental Daily Deaths",
     title="Chicago Ozone Concentration vs. Daily Deaths",
     legend=false,

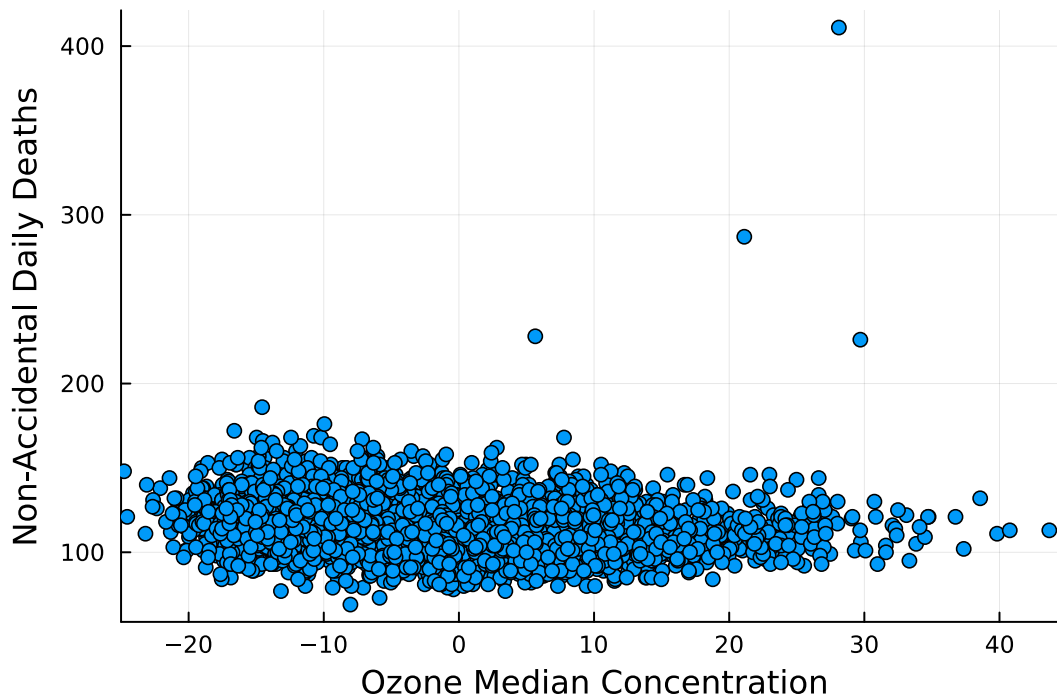
```

```

rightmargin=10Plots.mm,
leftmargin=5Plots.mm,
bottommargin=5Plots.mm
)
savefig("figs/1_1_o3.png")
plot!() #To show plot in notebook

```

Chicago Ozone Concentration vs. Daily Deaths

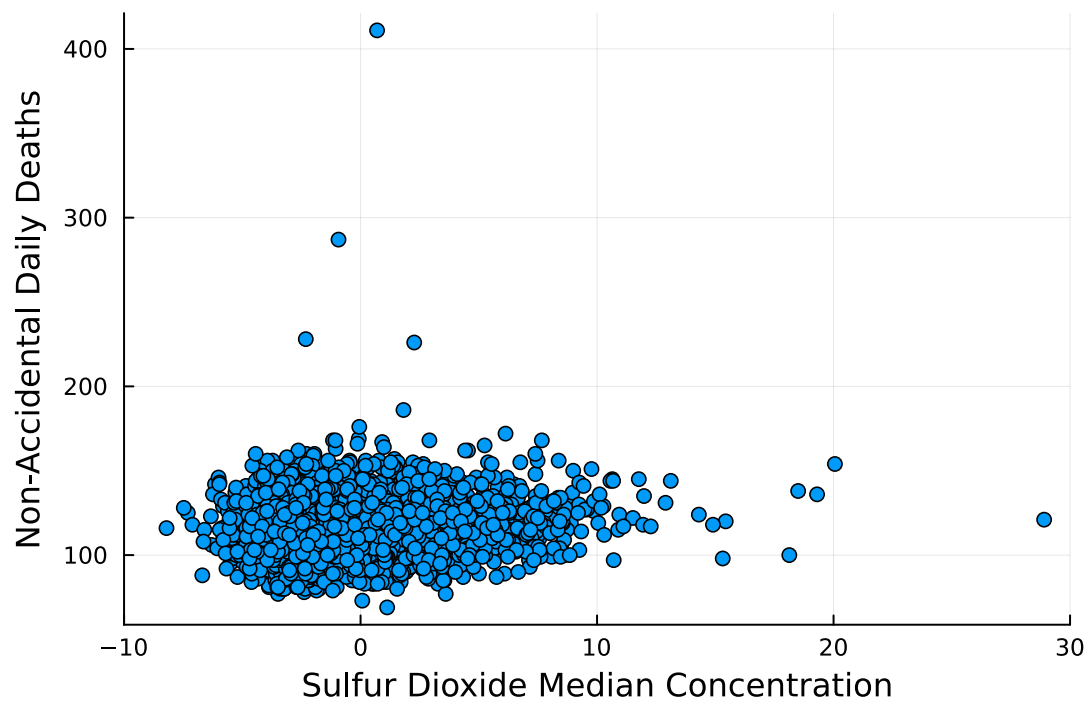


```

plot(chicago.so2median, chicago.death,
st="scatter", xlim=[-10,30],
xlabel="Sulfur Dioxide Median Concentration",
ylabel="Non-Accidental Daily Deaths",
title="Chicago Sulfur Dioxide Concentration vs. Daily Deaths",
legend=false,
rightmargin=10Plots.mm,
leftmargin=5Plots.mm,
bottommargin=5Plots.mm
)
savefig("figs/1_1_so2.png")
plot!() #To show plot in notebook

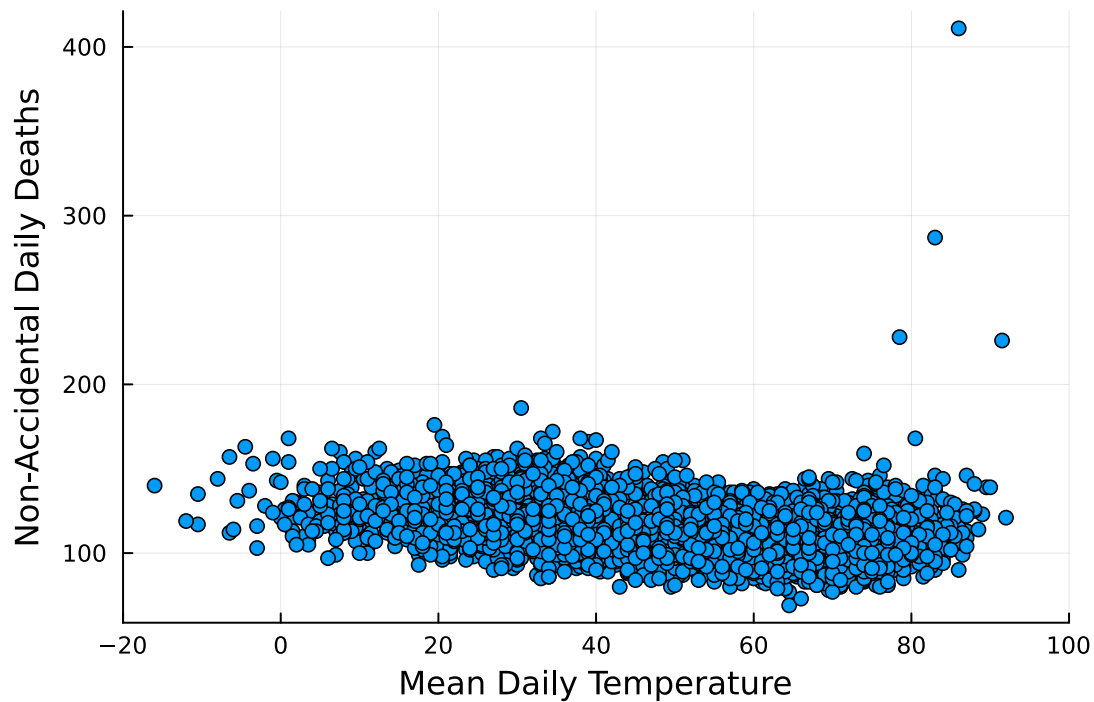
```

Chicago Sulfur Dioxide Concentration vs. Daily Deaths



```
plot(chicago.tmpd, chicago.death,  
     st="scatter", xlim=[-20,100],  
     xlabel="Mean Daily Temperature",  
     ylabel="Non-Accidental Daily Deaths",  
     title="Chicago Mean Daily Temperature vs. Daily Deaths",  
     legend=false,  
     rightmargin=10Plots.mm,  
     leftmargin=5Plots.mm,  
     bottommargin=5Plots.mm  
    )  
savefig("figs/1_1_temp.png")  
plot!() #To show plot in notebook
```

Chicago Mean Daily Temperature vs. Daily Deaths



1.2

```
X = chicago.tmpd
Y = chicago.death
beta = zeros(2)
beta[2] = cov(X,Y) / var(X)
beta[1] = mean(Y) - (beta[2]*mean(X))
@show beta
n = length(X)
sigma2 = (Y - [ones(n) X]*beta).^2
s2 = (n/(n-2))*(1/n)*sum(sigma2)
@show s2
```

```
beta = [129.95705121476823, -0.28964429615009885]
s2 = 202.31531425938184
```

```
202.31531425938184
```

```

#MLE Log-Likelihood Estimation
function chicago_loglik(theta, deaths, tmpd)
    beta0, beta1, sigma = theta
    mean = beta0 .+ (beta1*tmpd)
    ll = sum(logpdf.(Normal.(mean, sigma), deaths))
    return ll
end

lb = [0.0, -1000.0, 1.0]
ub = [1000.0, 1000.0, 100.0]
theta0 = [500.0, 0.0, 50.0]
optim_out = Optim.optimize(theta -> -chicago_loglik(theta, chicago.death, chicago.tmpd), lb,
theta_mle = optim_out.minimizer
@show theta_mle
sigma2_mle = (theta_mle[3]^2)*(n/(n-2))
# Numbers should match above cell
# They do to reasonable rounding
# yay

```

```
theta_mle = [129.9570512138531, -0.28964429613252857, 14.220977186157482]
```

```
202.3153142701072
```

1.3

```

# Simulate 10,000 predictions from data
x = -20:0.1:100 # every 0.1°F
mu = theta_mle[1] .+ theta_mle[2]*x

y_pred = zeros(length(x), 10000)
for i = 1:length(x)
    y_pred[i, :] = rand(Normal(mu[i], theta_mle[3]),10000)
end
y_q = mapslices(v -> quantile(v, [0.05, 0.5, 0.95]), y_pred; dims=2)

p = scatter(
    chicago.tmpd,
    chicago.death,
    xlabel="Mean Daily Temperature (°F)",
    ylabel="Daily Deaths",

```

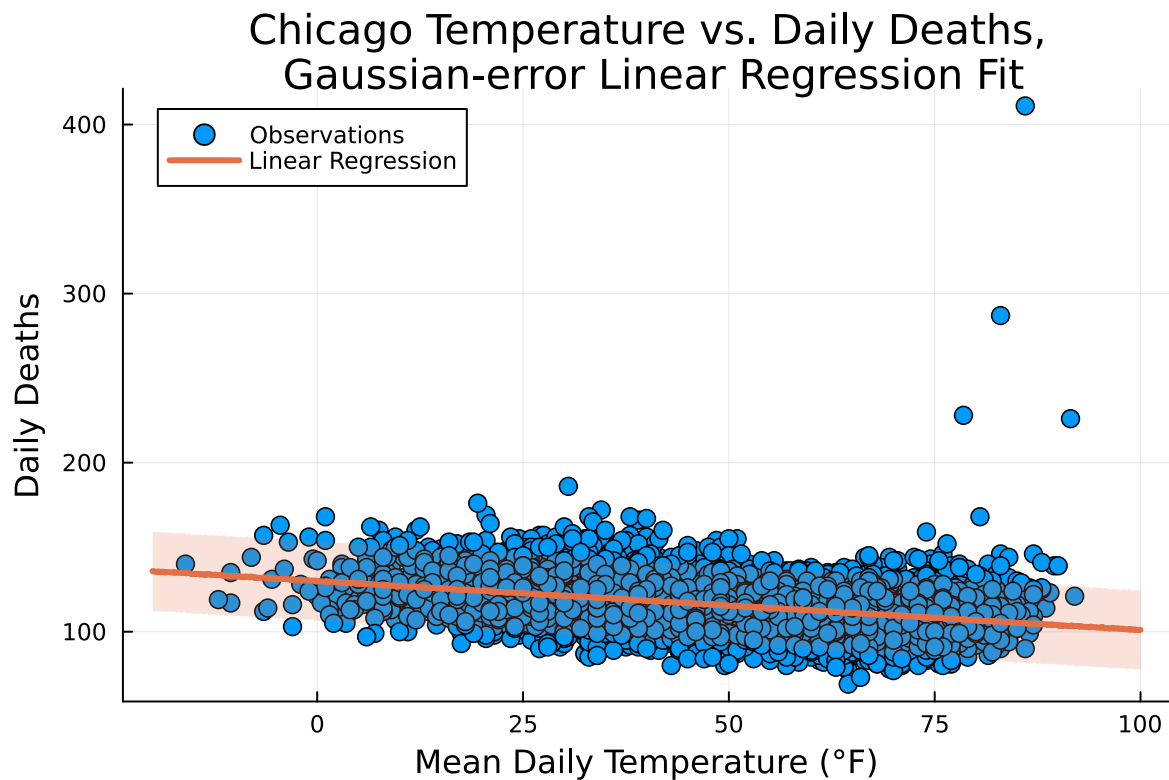
```

title="Chicago Temperature vs. Daily Deaths,\n Gaussian-error Linear Regression Fit",
markersize=5,
label="Observations",
size = (600, 400))

plot!(x,
      y_q[:, 2],
      ribbon = (y_q[:, 2] - y_q[:, 1], y_q[:, 3] - y_q[:, 2]),
      linewidth=3,
      fillalpha=0.2,
      label="Linear Regression",
      size=(600, 400))

savefig("figs/1_3_scatter.png")
plot!() #To show plot in notebook

```



```

residuals = Y .- (theta_mle[1] .+ (theta_mle[2].*X))
plot(residuals,
     st=:hist,

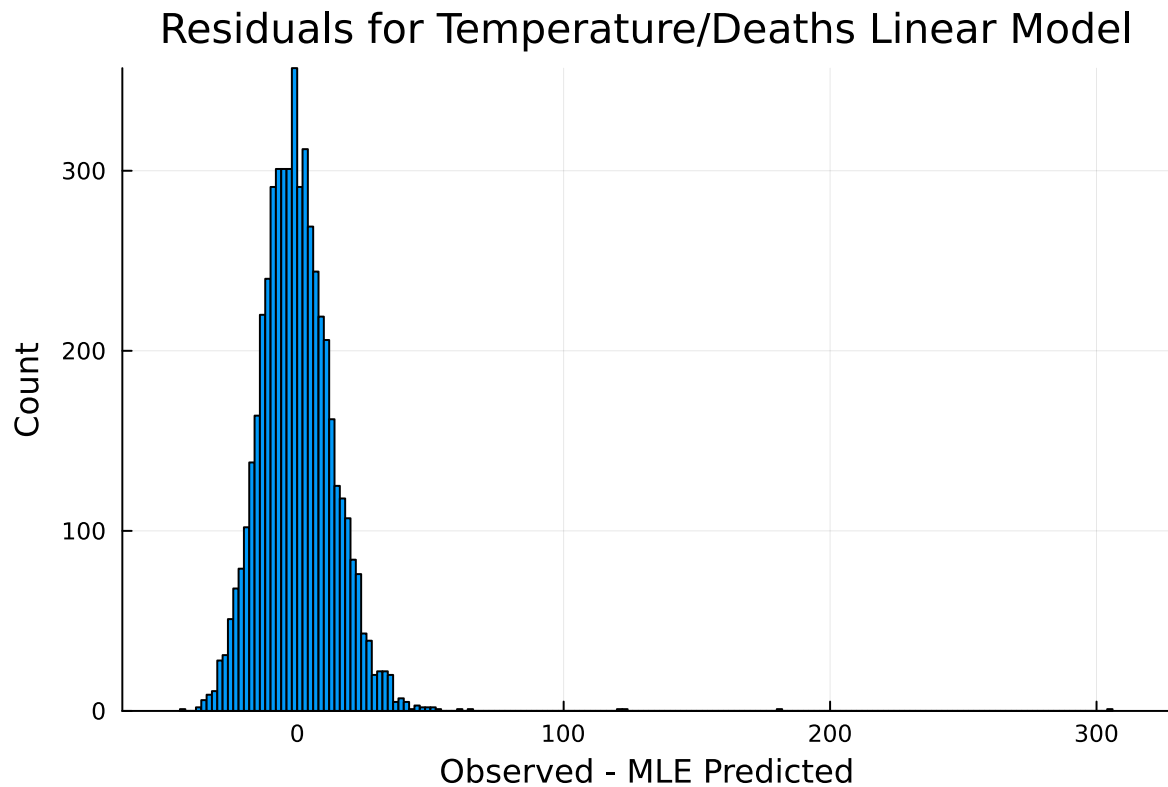
```



```

xlabel="Observed - MLE Predicted",
ylabel="Count",
title="Residuals for Temperature/Deaths Linear Model",
legend=false)
savefig("figs/1_3_hist.png")
plot!() #To show plot in notebook

```

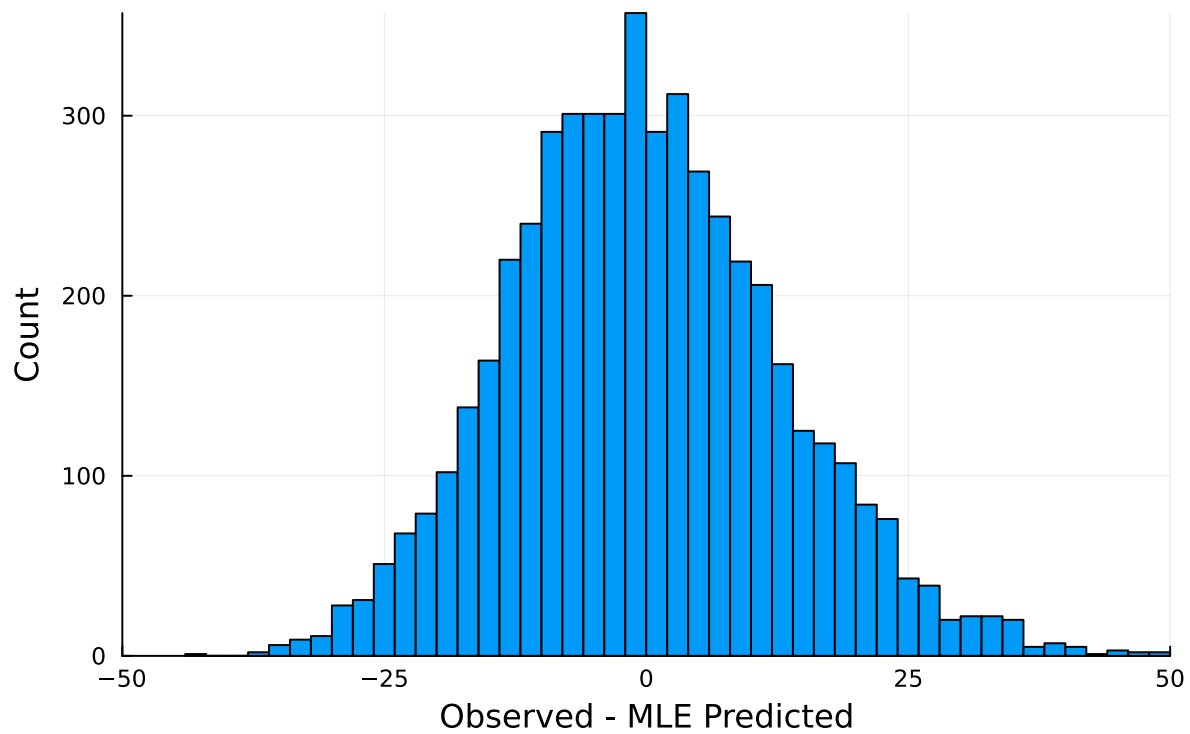


```

plot(residuals,
st=:hist,
xlabel="Observed - MLE Predicted",
xlim=(-50,50),
ylabel="Count",
title="Residuals for Temperature/Deaths Linear Model",
legend=false)
savefig("figs/1_3_zoom.png")
plot!() #To show plot in notebook

```

Residuals for Temperature/Deaths Linear Model



Problem 2

2.1

```
temp_dat = CSV.read("data/HadCRUT5.1Analysis_gl.txt", delim=" ",  
                    ignorerepeated=true,  
                    header=["Year", "January", "February", "March", "April", "May", "June", "July", "  
                    silencewarnings=true, DataFrame)  
temp_dat = temp_dat[3:2:nrow(temp_dat), :]
```

	Year	January	February	March	April	May	June	July	August	
	Int64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	
1	1851	-0.266	-0.467	-0.641	-0.528	-0.222	-0.207	-0.136	-0.148	...
2	1852	-0.346	-0.496	-0.567	-0.575	-0.113	-0.077	0.009	-0.149	...
3	1853	-0.204	-0.399	-0.273	-0.401	-0.27	-0.124	-0.072	-0.04	...
4	1854	-0.403	-0.392	-0.247	-0.327	-0.277	-0.302	-0.162	-0.233	...
5	1855	-0.17	-0.313	-0.345	-0.211	-0.368	-0.333	-0.315	-0.202	...
6	1856	-0.083	-0.347	-0.371	-0.314	-0.293	-0.225	-0.223	-0.249	...
7	1857	-0.494	-0.442	-0.488	-0.675	-0.654	-0.291	-0.404	-0.356	...
8	1858	-0.336	-0.488	-0.566	-0.43	-0.523	-0.36	-0.341	-0.288	...
9	1859	-0.405	-0.351	-0.322	-0.189	-0.17	-0.24	-0.29	-0.266	...
10	1860	-0.39	-0.478	-0.789	-0.31	-0.371	-0.213	-0.211	-0.137	...
11	1861	-0.905	-0.681	-0.422	-0.396	-0.697	-0.272	-0.349	-0.098	...
12	1862	-0.83	-0.797	-0.538	-0.533	-0.345	-0.275	-0.356	-0.5	...
13	1863	-0.087	-0.284	-0.414	-0.336	-0.391	-0.418	-0.357	-0.335	...
14	1864	-0.825	-0.605	-0.531	-0.557	-0.346	-0.116	-0.287	-0.344	...
15	1865	-0.23	-0.513	-0.717	-0.545	-0.495	-0.251	-0.297	-0.247	...
16	1866	-0.23	-0.472	-0.748	-0.549	-0.599	-0.13	-0.053	-0.291	...
17	1867	-0.501	-0.408	-0.712	-0.489	-0.616	-0.35	-0.259	-0.325	...
18	1868	-0.707	-0.711	-0.332	-0.384	-0.282	-0.298	-0.047	-0.195	...
19	1869	-0.304	0.137	-0.651	-0.412	-0.311	-0.396	-0.432	-0.178	...
20	1870	-0.214	-0.457	-0.495	-0.357	-0.304	-0.207	-0.083	-0.256	...
21	1871	-0.591	-0.688	-0.171	-0.222	-0.348	-0.27	-0.073	-0.324	...
22	1872	-0.594	-0.627	-0.519	-0.366	-0.121	-0.267	-0.109	0.033	...
23	1873	-0.237	-0.272	-0.39	-0.462	-0.319	-0.297	-0.158	-0.217	...
24	1874	-0.182	-0.429	-0.664	-0.422	-0.311	-0.556	-0.171	-0.287	...
...	...	...	...	...	...	...	...	...	...	...

```

temp_dat[:, "DJF_Mean"] = zeros(nrow(temp_dat))
temp_dat[:, "Frozen"] = zeros(nrow(temp_dat))

for i = 2:nrow(temp_dat)
    temp_dat[i, "DJF_Mean"] = mean([temp_dat[i-1, "December"], temp_dat[i, "January"], temp_dat[i, "February"]])
    if temp_dat.Year[i] in [1856, 1875, 1884, 1904, 1912, 1934, 1961, 1979, 2015]
        temp_dat[i, "Frozen"] = 1
    else
        temp_dat[i, "Frozen"] = 0
    end
end
end

```

```

function invlogit(x)
    return exp(x) / (1+exp(x))
end

```

```

function logistic_loglik(theta, y, x)
  beta_0, beta_1 = theta
  p = invlogit.(beta_0 .+ (beta_1*x))
  ll = sum(logpdf.(Bernoulli.(p), y))
  return ll
end

lb = [-5.0, -5.0]
ub = [5.0, 5.0]
p0 = [0.0, 0.0]
optim_out = Optim.optimize(p -> -logistic_loglik(p, temp_dat.Frozen .> 0, temp_dat.DJF_Mean)
mle = optim_out.minimizer

```

```

2-element Vector{Float64}:
-3.0178514758810584
-0.729619157916247

```

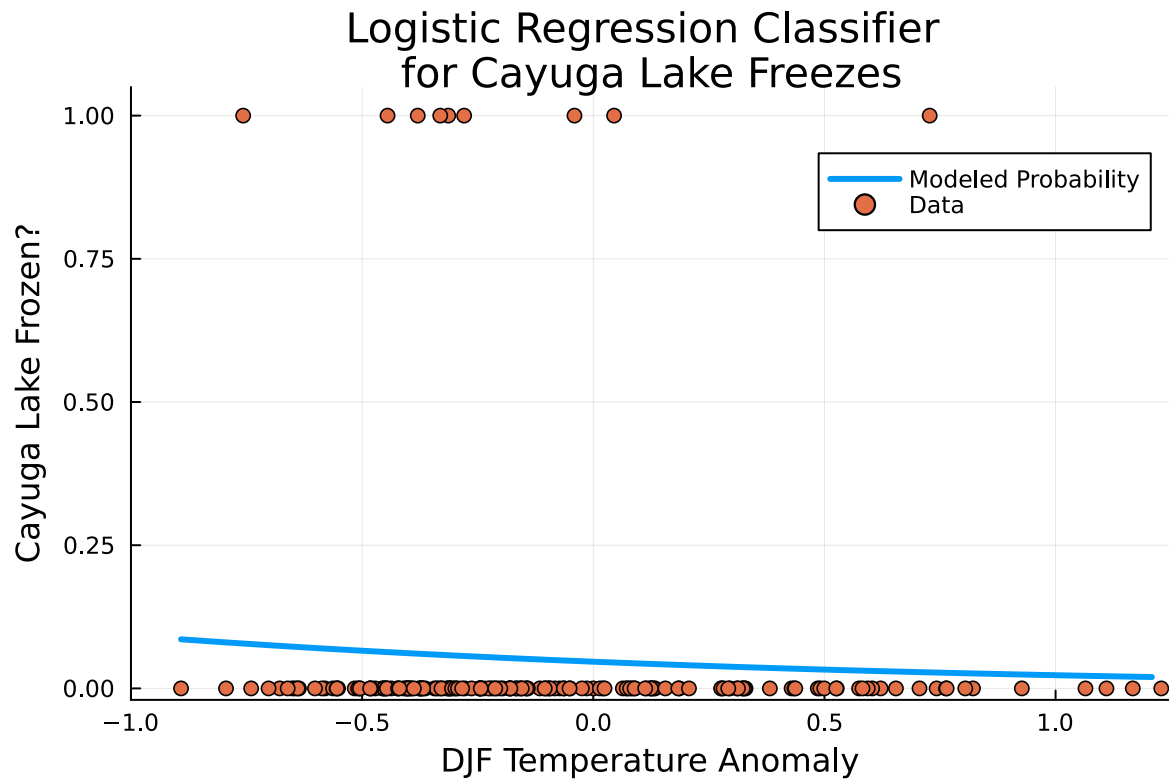
```
exp(mle[1]), exp(mle[2])
```

```
(0.04890618167442439, 0.4820925562668116)
```

```

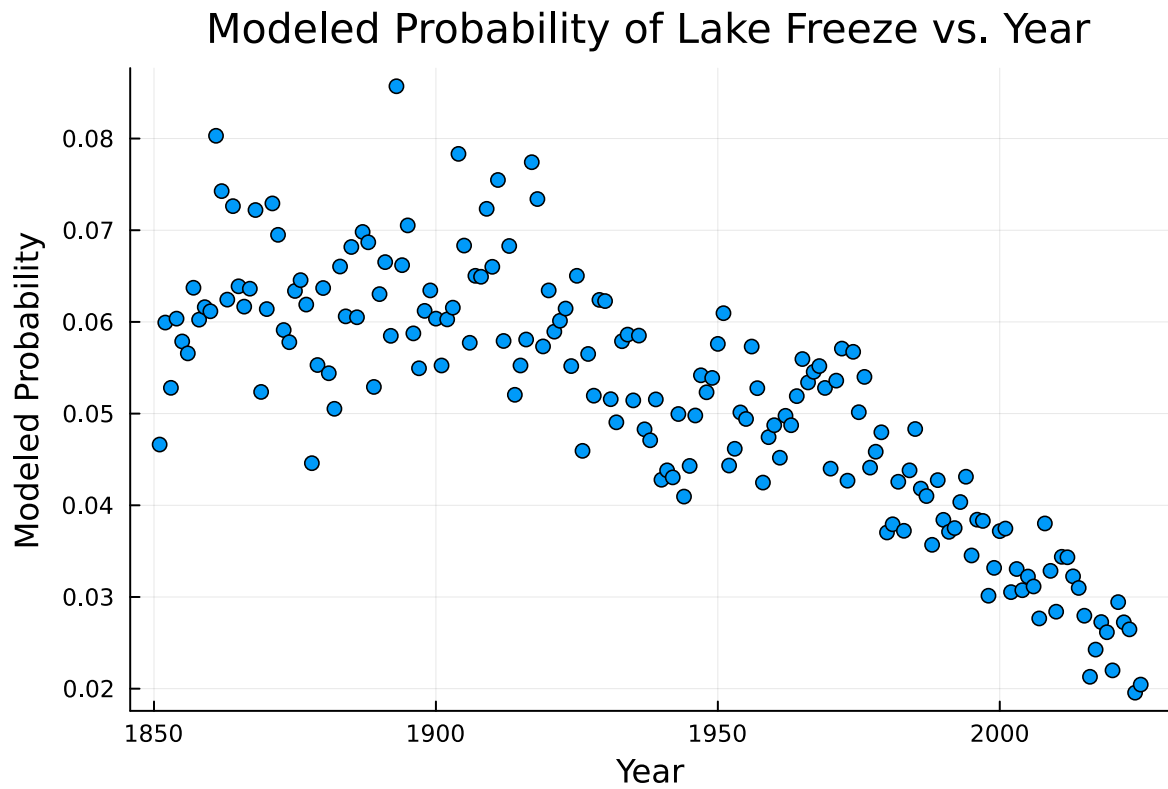
x_pred = minimum(temp_dat.DJF_Mean):0.1:maximum(temp_dat.DJF_Mean)
p_pred = invlogit.(mle[1] .+ mle[2] * x_pred)
plot(x_pred, p_pred, linewidth=3,
     label="Modeled Probability",
     xlabel="DJF Temperature Anomaly",
     ylabel="Cayuga Lake Frozen?",
     legend_position=(0.75,0.85),
     title="Logistic Regression Classifier \nfor Cayuga Lake Freezes",
     ylim=(-0.02,1.05), xlim=(-1, 1.25), size=(600, 400))
scatter!(temp_dat.DJF_Mean, temp_dat.Frozen .> 0, label="Data")
savefig("figs/2_1.png")
plot!() #To show plot in notebook

```



2.2

```
p_historical = invlogit.(mle[1] .+ mle[2]*temp_dat.DJF_Mean)
scatter(1851:2025, p_historical, legend=false,
        xlabel="Year", ylabel="Modeled Probability",
        title="Modeled Probability of Lake Freeze vs. Year")
savefig("figs/2_2.png")
plot!() #To show plot in notebook
```



2.3

```
(log(.01/.99) - mle[1])/mle[2]
```

2.1617694068753965

Problem 3

3.1

```
sals = CSV.read("data/salamanders.csv", delim=";", DataFrame)
```

	SITE	SALAMAN	PCTCOVER	FORESTAGE
	Int64	Int64	Int64	Int64
1	1	13	85	316
2	2	11	86	88
3	3	11	90	548
4	4	9	88	64
5	5	8	89	43
6	6	7	83	368
7	7	6	83	200
8	8	6	91	71
9	9	5	88	42
10	10	5	90	551
11	11	4	87	675
12	12	3	83	217
13	13	3	87	212
14	14	3	89	398
15	15	3	92	357
16	16	3	93	478
17	17	2	2	5
18	18	2	87	30
19	19	2	93	551
20	20	1	7	3
21	21	1	16	15
22	22	1	19	31
23	23	1	29	10
24	24	1	34	49
...	...	...	...	...

```

stdz(x) = (x .- mean(x)) / std(x)
function poisson_loglik(theta, grdcover, salcount)
    b0, b1 = theta
    lambda = exp.(b0 .+ (b1*grdcover))
    ll = sum(logpdf.(Poisson.(lambda), salcount))
    return ll
end

lb = [-100., -100.]
ub = [100., 100.]
th0 = [0., 0.]

optim_out = optimize(theta -> -poisson_loglik(theta, stdz(sals.PCTCOVER), sals.SALAMAN), lb,
mle = optim_out.minimizer

```

```
2-element Vector{Float64}:
 0.4295112911917795
 1.1595061319045692
```

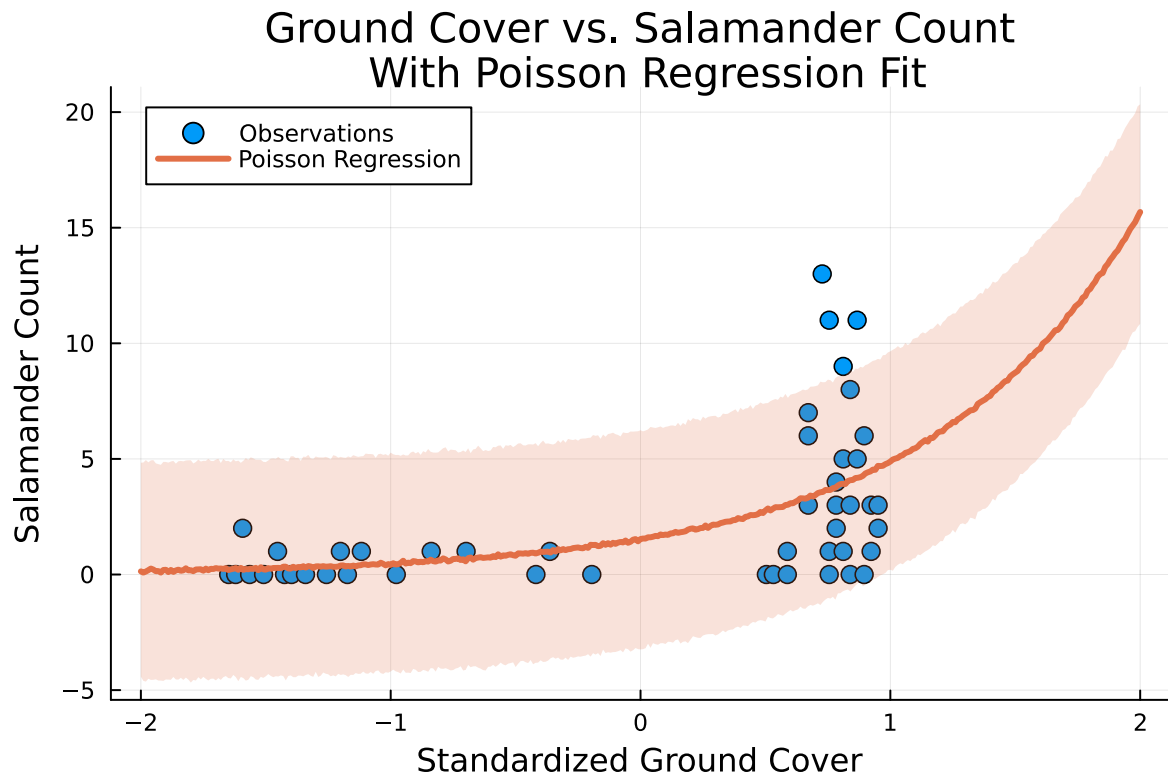
3.2

```
x_cover = -2:0.01:2
mu = exp.(mle[1] .+ (mle[2]*x_cover))
sigma2 = (1/nrow(sals))*sum((sals.SALAMAN - exp.(mle[1] .+ (mle[2]*stdz(sals.PCTCOVER))))).^2
sigma = sqrt(sigma2)

y_pred = zeros(length(x_cover), 10000)
for i = 1:length(x_cover)
    y_pred[i, :] = rand(Normal(mu[i], sigma), 10000)
end
y_q = mapslices(v -> quantile(v, [0.05, 0.5, 0.95]), y_pred; dims=2)

p = scatter(
    stdz(sals.PCTCOVER),
    sals.SALAMAN,
    xlabel="Standardized Ground Cover",
    ylabel="Salamander Count",
    title="Ground Cover vs. Salamander Count\n With Poisson Regression Fit",
    markersize=5,
    label="Observations",
    size = (600, 400))

plot!(x_cover,
    y_q[:, 2],
    ribbon = (y_q[:, 2] - y_q[:, 1], y_q[:, 3] - y_q[:, 2]),
    linewidth=3,
    fillalpha=0.2,
    label="Poisson Regression",
    size=(600, 400))
savefig("figs/3_2.png")
plot!() #To show plot in notebook
```

3.3

```
mse = sum((sals.SALAMAN - exp.(mle[1] .+ (mle[2]*stdz(sals.PCTCOVER))))).^2)/nrow(sals)
```

8.205326223735485

```
function poisson_loglik(theta, grdcover, forestage, salcount)
    b0, b1, b2 = theta
    lambda = exp.(b0 .+ (b1*grdcover) .+ (b2*forestage))
    ll = sum(logpdf.(Poisson.(lambda), salcount))
    return ll
end

lb = [-100., -100., -100.]
ub = [100., 100., 100.]
th0 = [0., 0., 0.]
```

```
optim_out = optimize(theta -> -poisson_loglik(theta, stdz(sals.PCTCOVER), stdz(sals.FORESTAGI
mle = optim_out$minimum
```

3-element Vector{Float64}:

0.4293187207669783

1.1624676136697254

-0.004084430274221949

```
mse = sum((sals.SALAMAN - exp.(mle[1] .+ (mle[2]*stdz(sals.PCTCOVER) .+ (mle[3]*stdz(sals.FO
```

8.204880991254662

Problem 4

4.1

```
hurricanes = CSV.read("data/Hurricanes.csv", DataFrame)
```

	name	year	deaths	category	min_pressure	damage_norm	female	femininity
	String15	Int64	Int64	Int64	Int64	Int64	Int64	Float64
1	Easy	1950	2	3	960	1590	1	6.77778
2	King	1950	4	3	955	5350	0	1.38889
3	Able	1952	3	1	985	150	0	3.83333
4	Barbara	1953	1	1	987	58	1	9.83333
5	Florence	1953	0	1	985	15	1	8.33333
6	Carol	1954	60	3	960	19321	1	8.11111
7	Edna	1954	20	3	954	3230	1	8.55556
8	Hazel	1954	20	4	938	24260	1	9.44444
9	Connie	1955	0	3	962	2030	1	8.5
10	Diane	1955	200	1	987	14730	1	9.88889
11	Ione	1955	7	3	960	6200	0	5.94444
12	Flossy	1956	15	2	975	1540	1	7.0
13	Helene	1958	1	3	946	540	1	9.88889
14	Debra	1959	0	1	984	430	1	9.88889
15	Gracie	1959	22	3	950	510	1	9.77778
16	Donna	1960	50	4	930	53270	1	9.27778
17	Ethel	1960	0	1	981	35	1	8.72222
18	Carla	1961	46	4	931	15850	1	9.5
19	Cindy	1963	3	1	996	300	1	9.94444
20	Cleo	1964	3	2	968	6450	1	7.94444
21	Dora	1964	5	2	966	16260	1	9.33333
22	Hilda	1964	37	3	950	2770	1	8.83333
23	Isbell	1964	3	2	974	800	1	9.44444
24	Betsy	1965	75	3	948	20000	1	8.33333
...	...	...	...	...	...	...	...	...

```
function poisson_loglik(theta, female, mslp, deaths)
    b0, b1, b2 = theta
    lambda = exp.(b0 .+ (b1*female) .+ (b2*mslp))
    ll = sum(logpdf.(Poisson.(lambda), deaths))
    return ll
end

lb = [-100., -100., -100.]
ub = [100., 100., 100.]
th0 = [0., 0., 0.]

optim_out = optimize(theta -> -poisson_loglik(theta, hurricanes.female, stdz(hurricanes.min_p),
mle = optim_out.minimizer
```

3-element Vector{Float64}:

```
2.414667236678301
0.47211173926789884
-0.7195324699539148
```

```
x_gen = -3:0.05:2
gen_male = exp.(mle[1] .+ (mle[2]*0) .+ (mle[3]*x_gen))
gen_female = exp.(mle[1] .+ (mle[2]*1) .+ (mle[3]*x_gen))
obs_male = hurricanes[hurricanes.female .== 0, :]
obs_female = hurricanes[hurricanes.female .== 1, :]

y_male = zeros(length(x_gen), 10000)
y_female = zeros(length(x_gen), 10000)
for i = 1:length(x_gen)
    y_male[i, :] = rand(Normal(gen_male[i], sqrt(var(hurricanes.deaths))),10000)
    y_female[i, :] = rand(Normal(gen_female[i], sqrt(var(hurricanes.deaths))),10000)
end
y_q_m = mapslices(v -> quantile(v, [0.05, 0.5, 0.95]), y_male; dims=2)
y_q_f = mapslices(v -> quantile(v, [0.05, 0.5, 0.95]), y_female; dims=2)

p = scatter(
    stdz(obs_male.min_pressure),
    obs_male.deaths,
    xlabel="Standardized Minimum Pressure",
    ylabel="Deaths",
    title="Minimum Pressure vs. Deaths\n With Poisson Regression Fit",
    markersize=5,
    label="Male Storms",
    size = (600, 400))

scatter!(
    stdz(obs_female.min_pressure),
    obs_female.deaths,
    color="#e77ceb",
    markersize=5,
    label="Female Storms",
    size = (600, 400))

plot!(x_gen,
    y_q_m[:, 2],
    ribbon = (y_q_m[:, 2] - y_q_m[:, 1], y_q_m[:, 3] - y_q_m[:, 2]),
    linewidth=3,
    fillalpha=0.2,
```

```

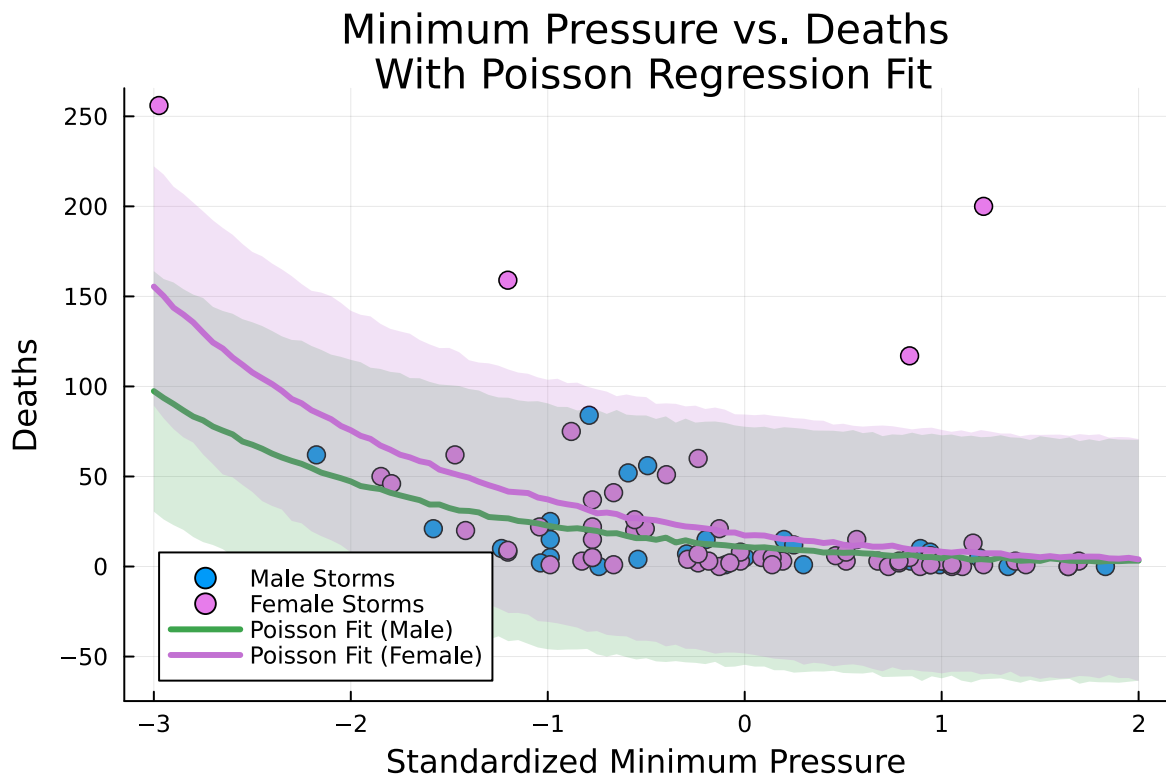
label="Poisson Fit (Male)",
size=(600, 400))

plot!(x_gen,
      y_q_f[:, 2],
      ribbon = (y_q_f[:, 2] - y_q_f[:, 1], y_q_f[:, 3] - y_q_f[:, 2]),
      linewidth=3,
      fillalpha=0.2,
      label="Poisson Fit (Female)",
      size=(600, 400))

savefig("figs/4_2.png")

plot!() #To show plot in notebook

```



```

scatter(
  obs_male.year,
  obs_male.deaths,

```

```

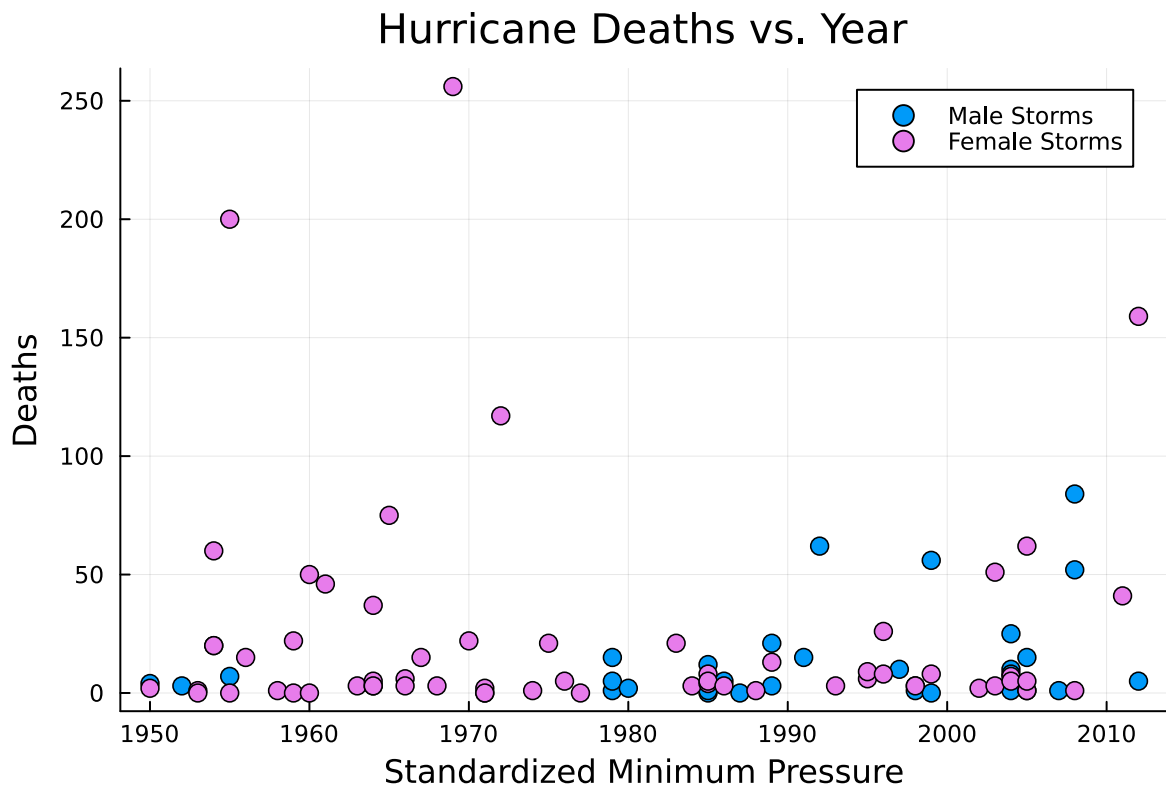
markersize=5,
label="Male Storms",
xlabel="Standardized Minimum Pressure",
ylabel="Deaths",
title="Hurricane Deaths vs. Year",
size = (600, 400))

scatter!(
  obs_female.year,
  obs_female.deaths,
  markersize=5,
  label="Female Storms",
  color="#e77ceb",
)

savefig("figs/4_4.png")

plot!() #To show plot in notebook

```



```
nrow(obs_female), nrow(obs_male)
```

```
(62, 30)
```